

Number: P3377R0

Date: 2026-05-12

Audience: [Evolution](#)

Target: C++29

Authors: [Hana Dusíková](#) and [Gašper Ažman](#)

- [Motivation](#)
 - [This is not removing `reinterpret\_cast`](#)
- [It's not just about compile-time programming](#)
- [Disclaimer](#)
- [Summary](#)
- [reinterpret_cast's use-cases](#)
 - [Diagnosable correctness](#)
 - [Casting between pointer and `uintptr\_t`](#)
 - [Changing the type of a function pointer](#)
 - [a named cast or `static\_cast`?](#)
 - [Changing type of a pointer-to-member](#)
 - [Data and function member together or separate?](#)
 - [Casting from a pointer to object to a pointer to byte/char](#)
 - [All you need is a cast](#)
 - [Casting from a first member of a standard-layout type to the enclosing object](#)
 - [Member pointer gives enough information](#)
 - [Example application](#)
 - [union's member pointer](#)
 - [Does this break pointer aliasing optimizations?](#)
 - [Conversion of a function pointer to/from object pointer](#)
- [Implementation](#)
 - [Pointer to integer roundtrip](#)
 - [Cast to enclosing object](#)
 - [Foreign objects in byte storage](#)
 - [Copying trivial foreign objects](#)
 - [Roundtripping function pointer and member pointers](#)
 - [Library or language?](#)
- [Wording](#)
 - [Feature test macro](#)

P3377R0: `constexpr reinterpret_cast` partial replacements

Currently C++ [doesn't allow `\[expr.const\]`](#) usage of `reinterpret_cast` during constant evaluation. This paper provides **equivalent** functionality.

First, it explores all valid use-cases of `reinterpret_cast`, and identifies a replacement API for each. It then shows how each can be implemented with a set of magic library functionality (and probably some tweeking of `[expr.const]`)

In the first revision, the paper proposes no wording. The aim is to explore and have a nod from EWG towards proposed design, if there is interest in doing this at all. Any wording will be provided in a later revision.

I want to thank everyone who did a review and gave me feedback: Bronek, Chandler, Daveed, Erich, Richard, Tomasz (in alphabet order).

Motivation

Over many years of the constexprification of C++, the committee have identified various blocking problems. This paper proposes features previously unavailable during constant evaluation (in "constexpr"). This will allow easy constexprification of at least `std::function`, `std::any`, and other polymorphic types in the standard library.

This is not removing `reinterpret_cast`

This paper doesn't aim to remove `reinterpret_cast`; it aims to provide bounded functionality which happens to work also during constant evaluation, in the same manner as `static_cast`, `const_cast` and `dynamic_cast` have done with c-style casts.

It's not just about compile-time programming

A huge side-effect of this proposal is improving the safety of C++ by making explicitly visible the programmer's intent, where previously she needed to use `reinterpret_cast` for multiple wildly different purposes.

As `reinterpret_cast` is defined to do various transformations, it's really easy to get confused what is supposed to be done just by looking at the code, like here in this snippet: `auto x = reinterpret_cast<T>(val);`. The programmer must be aware the compiler will pick a different transformations if the one she expects is not defined for her specific case.

One can even say `reinterpret_cast` is *"a swiss-army knife with multiple footguns and a minimal manual"*.

Disclaimer

All names are placeholders. Examples highly probably contain syntax errors.

Summary

functionality	runtime only (current status)	runtime & constexpr compatible replacement	status
---------------	----------------------------------	---	--------

pointer ↔ integer	<pre>intptr_t * iptr = reinterpret_cast<intptr_t> (ptr); auto * optr = reinterpret_cast<pointer> (iptr);</pre>	—	not proposed
pointer tagging	<pre>auto tptr = reinterpret_cast<pointer>(reinterpret_cast<intptr_t> (ptr) 0b1u); auto tag = static_cast<bool>(reinterpret_cast<intptr_t> (tptr) & 0b1u); auto optr = reinterpret_cast<pointer>(reinterpret_cast<intptr_t> (tptr) & ~0b1u);</pre>	<pre>auto tptr = tagged_pointer<int *, bool, 1> {ptr}; auto tag = tptr.tag(); auto optr = tptr.pointer();</pre>	P3125
casting to enclosing type	<pre>struct outer_t { int first; int second; }; outer_t outer{1,2}; auto * out1 = reinterpret_cast<outer_t *> (&outer.first); auto * out2 = reinterpret_cast<outer_t *> (&outer.second);</pre>	<pre>struct outer_t { int first; int second; }; outer_t outer{1,2}; auto & out1 = std::enclosing_cast<outer_t::first_member> (outer.first); auto & out2 = std::enclosing_cast<outer_t::second_member> (outer.second);</pre>	✓
accessing bytes of an object	<pre>auto * bytes = reinterpret_cast<std::byte *> (obj);</pre>	—	not proposed
integer / float shenanigans	<pre>double pi = 3.14; auto bits_of_pi = reinterpret_cast<int64_t>(pi); memcpy(bits_of_pi, pi, sizeof(pi)); // this is not UB</pre>	<pre>double pi = 3.14; auto bits_of_pi = std::bit_cast<uint64_t> (pi);</pre>	C++20
integer pointer shenanigans	<pre>int * iptr = ...; auto * uptr = reinterpret_cast<unsigned *> (iptr);</pre>	—	not proposed
using bytes as storage	<pre>char storage[N]; new(&storage[0]) T (...); T * ptr = reinterpret_cast<T *> (&storage[0]);</pre>	<pre>std::array<char, N> storage; new(storage.data()) T(...); T & ref = std::object_cast<T> (storage.data());</pre>	✓
copy object in byte-array	<pre>memcpy(other_storage, storage, N); T * first = std::start_lifetime_as<T> (&other_storage[0]); T * rest = reinterpret_cast<T*> (&other_storage[0]);</pre>	<pre>std::array other_storage = storage; // trivially copyable T T & all = std::object_cast<T> (other_storage.data());</pre>	✓

function ↔ object (pointers)	<pre>auto * optr = reinterpret_cast<T *>(fptr); // yes, really auto * fptr = reinterpret_cast<int(*)>()> (optr);</pre>	—	not proposed
type-erasing function pointer	<pre>auto * erased = reinterpret_cast<void(*)>()> (fp); int(*fptr)() = reinterpret_cast<int(*)>()> (erased);</pre>	<pre>auto erased = std::a_function_pointer{fp}; auto fptr = function_pointer_cast<int>()> (erased); auto fptr = static_cast<int(*)>(erased); // alt</pre>	✓
type-erasing member data pointer	<pre>struct empty_t {}; // yes you can do a member pointer of this auto * erased = reinterpret_cast<int (empty_t::*)>(mdp); auto mdp = reinterpret_cast<float (my_type::*)>(erased);</pre>	<pre>auto erased = std::a_member_data_pointer{mdp}; auto mdp = member_pointer_cast<float (my_type::*)>(erased); auto mdp = static_cast<float (my_type::*)> (erased); // alt</pre>	✓
type-erasing member function pointer	<pre>struct empty_t {}; // yes you can do a member pointer of this auto * erased = reinterpret_cast<void (empty_t::*)>(mfptr); auto mfptr = reinterpret_cast<int (my_type::*)>(float)>(erased);</pre>	<pre>auto erased = std::a_member_function_pointer{mp}; auto mfptr = member_pointer_cast<int (my_type::*)(float)>(erased); auto mfptr = static_cast<int (my_type::*) (float)>(erased); // alt</pre>	✓

reinterpret_cast's use-cases

I hope I found all possible and *valid* use-cases of `reinterpret_cast`, but it is possible there are more. The above table is not a list of things this paper proposes!

Diagnosable correctness

Each functionality provided by this paper has a specific, narrow, and unambiguous meaning, and fits C++'s existing abstract model. It won't expose underlying memory, or hardware behaviour, and is suitable for constant evaluation.

Casting between pointer and `uintptr_t`

Used for:

- pointer tagging (proposed in [P3125](#), currently in [LWG](#))
- passing pointers thru integer-taking low-level interfaces (not proposed)
- pointer arithmetic deliberately bypassing object size or alignment (not proposed)

The following example shows that even when conversion is allowed, it's really easy to mess up:

```
int value = ...;
static_assert(alignof(value) > 2);
int * tptr = reinterpret_cast<int *>(reinterpret_cast<uintptr_t>(&value) & 0b1);
// UB: tptr doesn't point to a valid object ✨
auto taddr = reinterpret_cast<uintptr_t>(&value) & 0b1; // this is fine

bool has_tag = bool{taddr & 0b1u}; // make sure which bit you mask

auto * iptr = reinterpret_cast<int *>(taddr & ~uintptr_t(0b1u)); // also fine, as
long you get original pointer and provide right bitmask
assert(iptr == &value);
```

Even when defined, when done explicitly, such manipulation is problematic when combined with pointer authentication. It also can't work on architectures without numeric addresses (like the C++ constant evaluator). The only portable way is asking the standard library to provide an implementation compatible with the target architecture.

[P3125](#) allows the following:

```
auto tptr = std::tagged_pointer<int, bool, 1>(&value); // tag is bool and we need
only one bit

bool has_tag = tptr.tag(); // it's already bool
int * iptr = tptr.pointer(); // it's already correct pointer

assert(iptr == &value);
```

Changing the type of a function pointer

Often used to pass a function through an interface as user-data via non-matching function pointer type, which is [safe as long as is not called](#). Another use-case is storing a function pointer in a type-erased interface.

```
int function();

int (*fptr)() = &function;
void (*opaque)() = reinterpret_cast<void(*)>(&fptr); // this is fine
opaque(); // once you call it, it's a terrible UB ✨
int (*other_side)() = reinterpret_cast<int(*)>(&opaque); // and we can go back
assert(other_side == &function);
other_side(); // and safely call it
```

A safe replacement needs to provide storage for a function pointer, whose size is implementation-defined. Therefore it will look like this:

```
int (*fptr)() = &function;
std::a_function_pointer opaque = fptr; // implicit cast from any function pointer
type
int (*other_side)() = function_pointer_cast<int()>(opaque) // fine as long as requested type is same (checkable)
```

Because the intent to type-erase is explicit, the compiler and library can provide a checked (at least at compile-time) interface and diagnose incorrect casts.

a named cast or static_cast?

The consequence of supporting `static_cast<T(As...)>` is that `std::a_function_pointer` must provide a conversion operator to function-pointer type. We must choose whether it should be marked explicit.

```
a_function_pointer fp = +[](int x) { printf("int: %d\n", x); };
void(*x)(int) = fp; // this does not work if explicit
takes_fptr(fp); // this also does not work if explicit :/
fp(42); // can't deduce return type, does not work at all
```

A named cast (`function_pointer_cast`) acts as if it were `static_cast` on an explicit conversion operator, except that it adds obviousness of purpose and customizability for user types under a purpose-directed name.

The authors therefore prefer a named cast: `function_pointer_cast<T()>` over `static_cast<T(*)>` for its explicitness. Supporting `static_cast` takes us one step closer to it being *swiss-army-knife-but-with-some-safeties-cast*.

Changing type of a pointer-to-member

Analogously, one can type-erase [pointers to class members](#). Pointers to class members have a different size than function pointers on most platforms (up to 24 bytes on 64-bit Windows). Usage is similar as the function-pointer roundtrip in the previous section, albeit with a different container type.

```

struct pair {
    int x;
    int y;
};

std::a_member_data_pointer opaque = &pair::x; // implicit cast from any pointer-to-
-data-member type
int (pair::*other_side) = member_pointer_cast<int(pair::*)>(opaque) // fine as lon
g as requested type is same (checkable)
assert(other_side == &pair::x);

instance.*other_side = 11;
assert(instance.x == 11);

```

Data and function member together or separate?

The question is whether we need to differentiate pointers to member functions and data members. One erasure type can easily make both work in polymorphic code without things like the infamous `nontype_t` of `std::function_ref` (which was a templated `constexpr` variable which provided a storage with a member pointer, so `function_ref` could reference it with `void *`).

The issue is that pointers to member functions can have a different size than pointers to data members, and making both work in `function_ref` would make the type larger and less useful. So, we learn from that example and propose separate types: `a_member_function_pointer` and `a_member_data_pointer`. They can share the named cast `member_pointer_cast`, since we can constrain the function template to the matching template argument types.

Casting from a pointer to object to a pointer to byte/char

Casts between storage types (arrays of `char`, `unsigned char`, and `std::byte`) and object types are common in serialization and deserialization of trivial types (where it is even valid to inspect their representation). For pointers, this is particularly brittle. We also see such casts when we explicitly manage storage, such as the short buffer optimization pattern in open polymorphism, or container internals, to store an object in properly sized and aligned byte storage.

```

template <size_t Size, size_t Alignment> struct alignas(Alignment) buffer_storage
{
    std::array<char, Size> buffer; // aligned at least as much as its enclosing object
    template <typename T> explicit buffer_storage(T && obj) requires(sizeof(T) >= Size && alignof(T) <= Alignment) {
        // I should also destroy lifetime of the buffer items, or make it uninitialized...
        // std::construct_at is already doing voidify, so it can take bytes, it will need a little tweaking
        std::construct_at<std::remove_cvref_t<T>>(buffer.data(), std::forward<T>(obj));
    }
    template <typename T> friend auto object_cast(auto && self) {
        // as long as the requested type is there
        return std::forward_like<decltype(self)>(reinterpret_cast<copy_cv_t<decltype(self), T*>>(buffer.data()));
    }
}

template <typename T> buffer_storage(T) -> buffer_storage<sizeof(T), alignof(T)>;

auto surprise = buffer_storage("happy birthday"s);
auto & str = object_cast<std::string>(surprise); // recall ""s is std::string

```

I rarely need access to object representations via `reinterpret_cast`. I *do* need the buffer example for open polymorphism. Currently such code can't be `constexpr`, so library authors opt for two different codepaths, one for runtime and one for constant evaluation.

At least in clang, the heap allocation in the constant evaluator costs about the same as any other value, so it's not a performance problem at compile-time, but it does make code look similarly as next example. Notice the `if constexpr` which access storage differently based on where the code is running. This makes such a polymorphic object unable to being transferred from constant evaluation to runtime, due missing non-transient allocations in C++.


```

template <size_t Size, size_t Alignment> struct alignas(Alignment) buffer_storage
{
    alignas(Alignment) union storage {
        std::array<char, Size> buffer;
        void * pointer;
    };

    template <typename T> constexpr friend auto object_cast(auto && self) -> auto&&{
        if constexpr {
            // now it's allowed static_cast only
            return std::forward_like<decltype(self)>(static_cast<copy_cv_t<decltype(self), T *>>(self.pointer));
        } else {
            return std::forward_like<decltype(self)>(reinterpret_cast<copy_cv_t<decltype(self), T *>>(buffer.data()));
        }
    }

    // ...
};

```

This approach can be mostly invisible from a library user perspective, but the inability to pass such object between runtime and compile time is problematic. We need to be able to use byte storages for placing other objects there.

All you need is a cast

The next example shows a replacement for this use-case of `reinterpret_cast`. It's very similar to the previous `object_cast`, but it takes anything convertible to a span of byte-like type:

```

template <typename T> constexpr auto object_cast(convertible_to_byte_span auto && bytes) -> auto&&{
    auto data = std::span{bytes};
    // Requirements:
    // - sizeof(T) <= data.size();
    // - is_aligned(data.data(), alignof(T)) == true
    // - T was already constructed there
    // Returns: a reference to object of type T which exists in memory inside `bytes`
    // Note: accessing content of `bytes` with pointer or reference of any other type
    // than T is ill-formed
    return std::forward_like<decltype(data)>(compiler_magic_cast<copy_cv_t<decltype(data)::value_type, T *>>(data));
}

```

This, together with a tweak to `construct_at`, is enough to provide `constexpr`-compatible short buffer optimization for polymorphic types (like `std::any`). Accessing values of bytes of storage so used must be not a constant expression, to not expose layout.

Casting from a first member of a standard-layout type to the enclosing object

Lewis made me aware of the following [affordance](#):

```
struct box_type {
    std::string name;
    std::string present;
};

const box_type & get_box_from_name(const std::string & name) {
    return *reinterpret_cast<const box_type *>(&name); // yes this is valid as far t
    he string points to a box_type::name
}

const box_type & get_box_from_present(const std::string & present) {
    return *reinterpret_cast<const box_type *>(&present); // this will never work fo
    r box_type::present
}
```

As you can see, the code is identical, it's the expectation that is different. This cannot be validated by a compiler at definition time. To achieve what `get_box_from_present` attempts to, some people want to use `offsetof` and then do [nonstandard pointer arithmetic](#) (also using `reinterpret_cast`):

```
box_type box = reinterpret_cast<box_type *>(reinterpret_cast<uintptr_t>(std::addressof(pres)) - offsetof(box_type, present))
```

Member pointer gives enough information

Previous example is perhaps fine in C, where there are no templates, `offsetof` is a pre-processor macro (and a dangerous one (!)), and polymorphism relies on common initial sequences. We achieve the same (and more) by replacing *one* `reinterpret_cast` with *two* `reinterpret_cast`s, which is not exactly an improvement; we also cannot use the pattern in generic code. But what if we built something like:

```

template <typename T> struct member_ptr_inspect;

template <typename Enclosing, typename Member> struct member_ptr_inspect<Member En
closing::*> {
    using enclosing_type = Enclosing;
    using member_type = Member;
};

template <typename T> using enclosing_type_of = member_ptr_inspect<T>::enclosing
_type;
template <typename T> using member_type_of = member_ptr_inspect<T>::member_type;

template <auto MemberPtr>
requires (MemberPtr != nullptr && std::is_standard_layout_v<enclosing_type_of<de
cltype(MemberPtr)>>) // later we can extend it with reflection annotation as opt-i
n
constexpr auto enclosing_cast(same_as<member_type_of<decltype(MemberPtr)>> auto &&
oref) -> copy_cvref_t<decltype(oref), enclosing_type_of<decltype(MemberPtr)>> {
    // Requirements:
    // - oref is reference to a submember (identified by MemberPtr) of enclosing_ty
pe_of<decltype(MemberPtr)>;
    // Returns: reference to enclosing type of oref
    // Postcondition: std::addressof(result.*MemberPtr) == std::addressof(oref)
}

```

Example application

Such interface doesn't merely give an explicit indication of what is converts to what, but it also makes it much easier for static analysis and the compiler to diagnose for incorrect usage, as is apparent in the following usage example:

```

constexpr const box_type & get_box_by_name(const std::string & name) {
    // yes this is valid as far the string points to a box_type::name
    return std::enclosing_cast<&box_type::name>(name);
}

constexpr const box_type & get_box_by_present(const std::string & present) {
    // yes this is valid as far the string points to a box_type::present (unlike befo
re when it never worked)
    return std::enclosing_cast<&box_type::present>(present);
}

```

Such casts are still unchecked, but the code shows intent and is easy to search for. It also allows exactly the stated operation, and not the *anything is possible* a reinterpret_cast signals. It's also arguably safer than naked pointer arithmetics. During constant evaluation, incorrect use can even be diagnosed.

For standard layout types, the operations this requires are already expressible in valid C++; for other types, we may choose to require an annotation on the parent that upcasts from pointers to members are allowed.

The validity of these operations was explored by Jeff Snyder in [P0149R0](#), but they were removed from the paper due to implementation complexity concerns. A proof-of-concept implementation was successful for the itanium and microsoft ABIs in a fork of clang by Hana.

union's member pointer

It's also possible to convert the pointer to member-of-union to the union itself, same as with an object's first member. This should transparently work with union types as well.

Does this break pointer aliasing optimizations?

No, compilers are already defensive in their optimizations in case someone does something naughty with `offsetof`, or already allowed a cast from first member. It's just not well known feature of C and C++ already.

Conversion of a function pointer to/from object pointer

It's possible to cast function pointer to `void *` (or even `T *`), both are conditionally supported as long as they fit. [This feels very troubling to the authors](#). This affordance is used when passing callbacks thru interfaces that take `void*`. As far as I'm aware, this is really not required for constexprification of any standard library functionality if we get other usages mentioned in this paper into C++.

Implementation

Working on it, did some prototypes for individual use-cases. I will try to have implementation ready for Brno, but it's unlikely. I mainly want to start a discussion. In following subsections I will describe how the implementation will work (based on discussion with various implementors and reading code of multiple compilers).

Pointer to integer roundtrip

Is not proposed by this paper and the pointer tagging functionality is proposed by [P3125](#) which does it by providing a value type which can be tracked by compiler, so compiler can also have metadata with the tag. It also has helpful information for a backend which can optimize on the information and keep information about pointer provenance.

Cast to enclosing object

In clang's constant evaluator, pointers are implemented as a rich abstraction containing path from object (stack / heap / static / ...) to a subobject, the member pointer must just match the last parts of the path, and if it matches, it's then dropped. Otherwise a diagnostics can be provided easily.

In more pointer-like pointer implementations an existing metadata about types needs to be used.

Foreign objects in byte storage

In a compiler like clang, a magic builtin for runtime will do exactly the same thing as this valid use-case of `reinterpret_cast` is already doing. For the constant evaluator, the compiler needs to mark `APValue` (holds any value: integer / pointer / object / array) of bytes in the array as **"holding a foreign value"**, at the address of first byte add a child containing an `APValue` with the real type of the object. When the `APValue` is accessed as a clang's `LValue`, a type mismatch can be diagnosed.

In a compiler like EDG which works with more real pointers with less abstraction, additional metadata with the actual type will need to be stored somewhere else, so it can be checked on access.

It's important that if the memory is accessed as a wrong type, compiler detect it and provide a diagnostic.

Copying trivial foreign objects

Simple objects without a custom constructor and no self-references, a simple `memcpy`-like approach works at runtime. At constant evaluation, the compiler already has all the information (in terms of metadata or special `APValue`), so one option is to allow `=default-ed` copy/move constructors to "just" work.

This unfortunately blocks usage of `vector` or any other type with a nontrivial copy constructor, and actually implements the copy on member-by-member basis. This can be solved in followup paper, perhaps something like this (inspired by `uninitialized_copy`):

```
constexpr void copy_with_enclosed_objects(std::span<const byte-like> source, std::span<byte-like> target) {
    if constexpr {
        // let compiler know it's something special (and maybe it needs to check metadata)
    } else {
        memcpy(target.data, source.data, source.size());
    }
}
```

Roundtripping function pointer and member pointers

This functionality can be provided in terms of using byte storage of this same proposal:

```

// hint how to make it checked in implementations:
template <typename T> static constexpr auto type_tag_of = type_tag_t{0};

struct a_function_pointer {
    std::array<char, sizeof(void(*)())> buffer;

    const type_tag_t * type_tag; // optional additional information for checking the
    type (in debug)

    constexpr a_function_pointer(const a_function_pointer & other) = default;

    template <function_pointer FPtr> constexpr a_function_pointer(FPtr fptr): type_tag{
    &type_tag_of<FPtr>} {
        static_assert(sizeof(fptr) <= buffer.size());
        new (buffer.data()) FPtr{fptr};
    }

    template <function_type FPtr> constexpr friend function_pointer_cast(same_as<a_function_pointer>
    auto && fp) pre(fp.type_tag == &type_tag_of<std::add_pointer_t<FPtr>>)) {
        // maybe additional check for
        return std::forward_like<decltype(fp)>(object_cast<std::add_pointer_t<FPtr>>(fp.buffer));
    }

    template <function_pointer FPtr> explicit constexpr operator FPtr(this auto && self)
    noexcept: type_tag{&type_tag_of<FPtr>} {
        // to support a static_cast<void(*)()>>(afptr);
        return object_cast<FPtr>(self.buffer);
    }
};

struct a_member_data_pointer {
    std::array<char, ??> buffer; // implementors knows what to put there

    constexpr a_member_data_pointer(const a_member_data_pointer & other) = default;

    template <member_data_pointer MDPtr> constexpr a_member_data_pointer(MDPtr mdptr):
    type_tag{&type_tag_of<MDPtr>} {
        static_assert(sizeof(MDPtr) <= buffer.size());
        new (buffer.data()) MDPtr{mdptr};
    }

    template <member_data_pointer MDPtr> constexpr friend member_pointer_cast(a_member_data_pointer
    fp) pre(fp.type_tag == &type_tag_of<MDPtr>)) {
        // maybe additional check for
        return object_cast<MDPtr>(fp.buffer);
    }

    template <member_data_pointer MDPtr> explicit constexpr operator MDPtr(this auto && self)
    noexcept: type_tag{&type_tag_of<MDPtr>} {
        // to support a static_cast<void(type::*)>>(mdptr);

```

```

    return object_cast<MDPtr>(self.buffer);
}
}

struct a_member_function_pointer {
    std::array<char, ??> buffer; // implementors knows what to put there

    constexpr a_member_function_pointer(const a_member_function_pointer & other) = default;

    template <member_function_pointer MFPtr> constexpr a_member_function_pointer(MFPtr MFPtr): type_tag{&type_tag_of<MFPtr>} {
        static_assert(sizeof(MFPtr) <= buffer.size());
        new (buffer.data()) MFPtr{MFPtr};
    }

    template <member_function_pointer MFPtr> constexpr friend member_function_pointer _cast(a_member_function_pointer fp) pre(fp.type_tag == &type_tag_of<MFPtr>) {
        // maybe additional check for
        return object_cast<MFPtr>(fp.buffer);
    }

    template <member_function_pointer MFPtr> explicit constexpr operator MFPtr(this auto && self) noexcept: type_tag{&type_tag_of<MFPtr>} {
        // to support a static_cast<void(type::*)()>>(mdptr);
        return object_cast<MFPtr>(self.buffer);
    }
}

```

Library or language?

Question is `a_function_pointer` and `a_member_data_pointer` should be defined in library or language. They should have same size and layout as a function pointer and a member pointer, so they can be `reinterpret_cast`-ed in an existing generic code.

Putting this into language would allow us to easily make things like `std::is_pointer_v<a_function_pointer>` to be true. Which would make a generic code easy to work with. I don't really have a strong opinion, and the implementation example is mostly about showing inner workings, not exactly how it should work.

Wording

First discuss in EWG if we want this design, and if so most of the wording will be in LEWG with tiny changes in `[expr.const]`.

Feature test macro

15.11 Predefined macro names `[cpp.predefined]`

__cpp_hana_cast 2026??